

How to Build an AI Agent

From Prompted LLMs to Reliable Agentic Systems

Diogo Ribeiro

ESMAD - Escola Superior de Média Arte e Design

Lead Data Scientist, Mysense.ai

✉ dfr@esmad.ipp.pt  0009-0001-2022-7072

May 22, 2026

Outline

- 1 What an AI Agent Is
- 2 System Architecture
- 3 Building Blocks
- 4 Reliability and Safety
- 5 From Prototype to Production

What an AI Agent Is

Agent vs. Single LLM Call

Single LLM call:

- One prompt in, one response out
- No persistent state
- No autonomous tool use

AI agent:

- Goal-directed loop: plan, act, observe, adapt
- Uses tools (search, code, APIs, databases)
- Maintains memory and context over time
- Handles failures and retries

Core Agent Loop

Agent Control Loop

Repeat until done:

1. Interpret user goal and current state
2. Decide next action
3. Execute tool or model call
4. Evaluate outcome against success criteria
5. Update memory and continue or stop

Design implication

The loop logic is software engineering. The LLM is one component, not the whole system.

System Architecture

Reference Architecture

- **Orchestrator:** state machine and policy logic
- **Model layer:** LLM(s) for reasoning and generation
- **Tool layer:** deterministic actions (APIs, DB, shell, workflows)
- **Memory layer:** short-term context and long-term retrieval
- **Safety layer:** permissions, validation, guardrails
- **Observability layer:** logs, traces, quality metrics, cost

Choose the Right Agent Pattern

Simple Router

- Fast, low complexity
- Great for FAQ and triage

Tool-Using ReAct Agent

- Think-act-observe loops
- Works for research and operations

Planner + Executor

- Better for long tasks
- Easier to checkpoint

Multi-Agent Workflow

- Specialized roles
- Higher coordination overhead

Building Blocks

Tool Interface Contract

Keep tool calls structured and typed.

```
1 {  
2   "name": "get_customer_invoice",  
3   "description": "Fetch invoice by id",  
4   "input_schema": {  
5     "type": "object",  
6     "properties": {  
7       "invoice_id": {"type": "string"}  
8     },  
9     "required": ["invoice_id"]  
10  }  
11 }
```

Listing 1: Minimal tool schema

- Validate input before execution
- Return explicit success/error payloads

Use layered memory, not one giant prompt.

- **Working memory:** current task context
- **Session memory:** recent interactions and outcomes
- **Long-term memory:** indexed facts, policies, and user preferences

Common failure mode

Storing everything in conversation history causes context drift, latency, and hallucinations.

Planning and Decomposition

1. Convert vague intent into explicit success criteria
2. Split task into atomic steps with clear dependencies
3. Execute smallest safe next action
4. Re-plan when observations diverge from expectations

Example

User goal: "Prepare Q2 churn analysis"

Agent plan: fetch data → validate schema → run analysis → generate slides → request approval.

Reliability and Safety

Guardrails You Actually Need

- **Permission boundaries:** role-based tool access
- **Input/output validation:** schema + policy checks
- **Human-in-the-loop gates:** approvals for destructive actions
- **Timeouts and retries:** bounded recovery behavior
- **Fallback paths:** deterministic flows when agent confidence is low

Evaluate offline and in production. Quality metrics

- Task success rate
- Tool-call precision
- Hallucination rate
- Recovery rate after failure

Operational metrics

- Latency per step
- Cost per task
- Token consumption
- Escalation frequency

From Prototype to Production

Implementation Roadmap

1. Start with one narrow, high-value workflow
2. Implement deterministic tools first
3. Add a constrained agent loop with strict schemas
4. Instrument traces and evaluate against test tasks
5. Add memory and planner only when baseline is stable
6. Expand scope after reliability targets are met

Common Anti-Patterns

- Building a general autonomous agent too early
- Missing tool contracts and error taxonomies
- No golden test set for regressions
- Prompt-only fixes for architectural problems
- Ignoring cost controls until deployment

Pragmatic rule

If behavior must be guaranteed, move it into deterministic code, not prompt text.

Key message

A production AI agent is a software system with an LLM component, not an LLM prompt with extra steps.

Checklist:

- Clear task boundaries
- Typed tool interfaces
- Explicit state and memory strategy
- Guardrails and approval gates
- Continuous evaluation and monitoring